

Deep Reinforcement Learning with Specular Reflection Control for Mobile Robot Collision Avoidance Based on Multi-Directional Distance Perception

Abstract—Safe autonomous navigation in cluttered environments remains a fundamental challenge for mobile robots. This paper presents a two-stage framework that combines supervised collision prediction with deep reinforcement learning (DRL) for collision avoidance, centered on a physics-inspired specular reflection control mechanism. A Pygame-based 2D simulation environment is constructed, in which a mobile robot is equipped with nine multi-directional range sensors covering a $\pm 120^\circ$ field of view. First, a multi-layer perceptron (MLP) is trained to predict future collisions within a 30-frame horizon using 11-dimensional sensor-motion features, achieving an accuracy of 83.65% and a recall of 84.08% on held-out test episodes. Second, the collision avoidance problem is formulated as a Markov decision process with a 14-dimensional state space and two discrete actions, where a deep Q-network (DQN) learns when to initiate an avoidance maneuver while the how is handled by a deterministic three-phase pipeline: deceleration, specular reflection, and re-acceleration. The reflection direction is computed geometrically from the incident heading vector and the obstacle surface normal via $\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$. We conduct systematic model iterations across five versions (V1 through V4 and V3-Shield), examining the effects of reflection normal extraction, reward function design, state representation, dynamic braking distance, and external safety intervention. Comprehensive experiments demonstrate that the final V3 DQN policy achieves a collision rate of 1.20% with a survival rate of 98.80% over 500 blind-test episodes, substantially outperforming earlier versions and baseline strategies. Notably, our results reveal that increasing model complexity and adding external safety rules do not necessarily improve performance—V4 (17-dimensional state with dynamic braking) and V3-Shield (external safety patch) both exhibit higher collision rates than V3. This finding underscores the importance of maintaining a stable coupling between perception, learned decision-making, and geometric control, rather than adding unvalidated safety heuristics on top of a trained policy.

Index Terms—collision avoidance, deep reinforcement learning, DQN, specular reflection, robot navigation, collision prediction, multi-sensor perception

I. Introduction

With the rapid advancement of intelligent manufacturing, warehouse logistics, public services, and specialized field operations, mobile robots are increasingly transitioning from structured environments to unstructured settings characterized by random obstacles, complex boundaries, and uncertain disturbances [1]. During autonomous operation, robots must continuously adjust their motion based

on limited perceptual information while avoiding collisions with walls, equipment, and other obstacles. Collisions not only disrupt task execution but may also cause equipment damage and pose safety risks. Therefore, accurately identifying potential collision risks and executing timely, effective avoidance maneuvers are critical problems in autonomous robot navigation and safety control.

Traditional approaches to collision avoidance include artificial potential field methods [2], vector field histograms (VFH) [3], and the dynamic window approach (DWA) [4]. These methods typically rely on range sensors such as LiDAR or ultrasonic arrays to obtain relative positions between the robot and obstacles, generating avoidance actions based on manually specified safety thresholds, potential functions, or local path planning rules. While physically interpretable and computationally efficient, their control performance often depends heavily on safety thresholds and environmental assumptions. When the robot operates at high speed, when obstacles fall within sensor ray gaps, or when the environmental geometry changes, decisions based solely on instantaneous minimum range readings may fail to adequately capture the true collision risk.

Recent advances in machine learning have opened new possibilities. Supervised learning models can extract nonlinear relationships between sensor states and collision outcomes from historical motion data, enabling anticipatory collision forecasting [13]. Deep reinforcement learning (DRL), particularly Deep Q-Networks (DQN) [7], Proximal Policy Optimization (PPO) [8], and Soft Actor-Critic (SAC) [9], allows robots to learn action policies through repeated interaction with the environment under the guidance of reward feedback [11], [12]. Compared to fixed-threshold methods, DRL does not require explicit control rules for every obstacle configuration and offers a degree of environmental adaptability.

However, if a DRL model is required to simultaneously control speed, steering angle, and avoidance timing, the continuous action space significantly increases training difficulty and may lead to policy instability and opaque control processes [10]. To address this, we adopt a hybrid approach that combines learned decision-making with ge-

ometric control: the DQN is responsible solely for deciding whether to initiate an avoidance maneuver, while the specific reflection direction is determined by the obstacle surface normal and the law of specular reflection. This design reduces the DRL action space to two discrete actions and preserves the interpretability of geometric control.

Our main contributions are as follows:

- 1) We design a nine-sensor range-finding system with surface normal extraction that provides rich environmental perception for both collision prediction and avoidance.
- 2) We propose a three-phase motion control pipeline (cruise $\rightarrow$ braking $\rightarrow$ specular reflection $\rightarrow$ re-acceleration) and formulate it as a discrete-action DQN problem, where the network learns optimal maneuver timing while the reflection direction follows a deterministic geometric law.
- 3) We conduct systematic model iterations across five versions (V1 through V4 and V3-Shield), examining how reflection normal extraction, reward function design, state representation, dynamic braking, and external safety intervention affect avoidance performance.
- 4) Through extensive experiments including ablation studies and strict paired blind tests, we demonstrate that the V3 policy achieves a 1.20% collision rate and, critically, that increasing model complexity or adding unvalidated external safety rules can counterproductively degrade performance.

II. Background and Related Work

A. Collision Avoidance in Mobile Robotics

Collision avoidance has been studied extensively in the robotics literature. The artificial potential field method [2] models obstacles as repulsive forces and goals as attractive forces, guiding the robot through gradient descent. However, this approach suffers from local minima and oscillatory behavior in narrow passages. The Vector Field Histogram (VFH) [3] and its variants [5] construct polar histograms of obstacle density and select steering directions that minimize collision risk while maintaining progress toward the goal.

The Dynamic Window Approach (DWA) [4] searches over admissible velocity commands, evaluating each candidate based on clearance, heading alignment, and speed. DWA remains widely used in practice due to its real-time performance, but its reactive nature limits performance in cluttered environments requiring anticipatory planning [6]. More generally, rule-based approaches tend to be brittle: their effectiveness degrades when the environment deviates from the assumptions under which the rules were designed.

B. Deep Reinforcement Learning for Navigation

The seminal Deep Q-Network (DQN) [7] demonstrated that neural networks can learn effective policies directly from high-dimensional sensory inputs. Subsequent work extended DRL to continuous control [8], [10] and robotics domains.

In the context of robot navigation, Tai et al. [11] trained a DQN-based controller for mapless motion planning using sparse 2D laser readings. Zhu et al. [12] combined DRL with imitation learning for target-driven visual navigation. Kahn et al. [13] proposed self-supervised DRL for collision avoidance using monocular images. Long et al. [14] developed a PPO-based planner for multi-robot collision avoidance. The Stable-Baselines3 library [15] and Gymnasium interface [16] have become standard tools for DRL research, providing reliable implementations and standardized environment APIs.

Despite these advances, most DRL navigation systems treat collision avoidance as an emergent behavior without explicit geometric reasoning about reflection angles, leading to suboptimal trajectories and unnecessary detours. Our work differs by embedding the law of specular reflection directly into the action execution pipeline, allowing the DRL agent to focus on when to maneuver rather than how.

C. Specular Reflection in Physics and Graphics

The law of specular reflection states that the angle of incidence equals the angle of reflection relative to the surface normal. In computer graphics, this is expressed as $\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$, where $\mathbf{d}$ is the incident direction vector and $\mathbf{n}$ is the unit surface normal. We adapt this principle to robot navigation, using obstacle surface normals extracted from ray-casting sensors to compute natural avoidance directions. This approach yields geometrically parsimonious trajectories that minimize unnecessary deviation from the original path.

III. Method

A. Environment and Sensor System

The simulation environment is implemented using Pygame with a window size of 1000×700 pixels. The environment contains four rectangular obstacles of varying sizes and aspect ratios, embedded within a bounded arena with walls of thickness $W_t = 5$ pixels. The robot is modeled as a circular agent with radius $R = 22$ pixels. Each episode begins with the robot placed at a random collision-free position and orientation, and terminates either upon collision or after 600 time steps.

1) **Directional Range Sensors:** The robot is equipped with nine directional range sensors uniformly distributed at angles $\{-120^\circ, -90^\circ, -60^\circ, -30^\circ, 0^\circ, 30^\circ, 60^\circ, 90^\circ, 120^\circ\}$ relative to the robot's forward direction. Each sensor casts a ray of maximum length $S_{\max} = 160$ pixels and returns the

Euclidean distance to the nearest obstacle intersection. Rays that do not intersect any obstacle return $S_{\max}$.

Critically, each ray also computes the unit surface normal vector $\mathbf{n}$ at the intersection point. When a ray hits a corner (intersection of two edges), the normals from both edges are averaged and normalized, producing a smooth direction for corner reflection. This multi-directional sensor array simultaneously captures the local obstacle distribution in front of, to the sides of, and behind the robot, providing rich environmental context for both collision prediction and DRL control.

2) Collision Detection: Collision between the circular robot and rectangular obstacles is determined by finding the nearest point on each rectangle to the robot center and checking whether the Euclidean distance is less than or equal to the robot radius R . Boundary collisions are checked against the four arena walls.

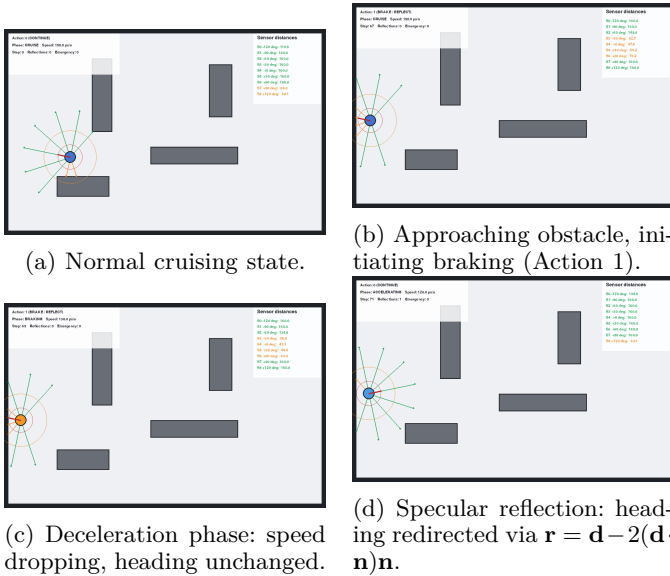


Fig. 1: Simulation environment and the three-phase collision avoidance sequence. The robot (blue circle) is equipped with nine range sensors (colored rays: green = safe, orange = warning, red = emergency). Four rectangular obstacles and boundary walls are shown in dark gray. The yellow and red rings indicate the warning ($D_w = 90$ px) and emergency ($D_e = 42$ px) distance thresholds.

B. Collision Prediction via MLP

1) Data Collection and Labeling: We collect raw trajectory data through the simulation environment in automatic mode, where the robot executes randomly sampled motion primitives (forward, forward-left, forward-right, turn-left, turn-right, backward) with varying durations. Each time step records the nine sensor readings, linear and angular velocities, robot pose, and a binary collision indicator. The raw dataset contains 5,221 records. After excluding the final 30 frames of each episode (for which a

full future window cannot be constructed), 5,160 valid samples remain, with 3,729 non-collision samples and 1,431 collision samples (positive ratio: 27.73%).

For supervised learning, we generate binary labels indicating whether a collision will occur within the next H frames (with $H = 30$). Formally, for each time step t with collision event indicator $c_t \in \{0, 1\}$, the label is:

$$y_t^{(H)} = \max\{c_{t+1}, c_{t+2}, \dots, c_{t+H}\} \quad (1)$$

Frames where a collision has already occurred ($c_t = 1$) are excluded from the training set, as the predictor should operate on pre-collision states.

Since consecutive frames within the same episode are highly correlated, random shuffling at the frame level would cause data leakage. We therefore split the data by complete episodes into 70% training, 15% validation, and 15% test sets using group-aware shuffling [15].

2) Model Architecture: The collision predictor is a multi-layer perceptron (MLP) with the following architecture:

$$\text{MLP} : \mathbb{R}^{11} \rightarrow \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^{32} \xrightarrow{\text{ReLU}} \mathbb{R}^1 \quad (2)$$

The 11-dimensional input consists of the nine sensor distances, linear speed, and angular speed. The model is trained with binary cross-entropy loss with class-balanced positive weighting to address label imbalance:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [w_p y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] \quad (3)$$

where $w_p = N_{\text{neg}}/N_{\text{pos}}$ is the ratio of negative to positive samples. The final classification threshold is set to 0.55 after optimization on the validation set.

C. DQN-Based Collision Avoidance

1) State and Action Spaces: The reinforcement learning environment is implemented as a Gymnasium environment. The state space $\mathcal{S} \subset \mathbb{R}^{14}$ (for V1–V3) comprises:

- $s_{0:8}$: Nine sensor distances normalized to $[0, 1]$
- s_9 : Current speed normalized by $v_{\text{normal}} = 150$ px/s
- s_{10} : $\sin(\theta)$ of the robot heading angle
- s_{11} : $\cos(\theta)$ of the robot heading angle
- s_{12} : Binary indicator for the braking phase
- s_{13} : Binary indicator for the accelerating phase

The sine and cosine representation of heading avoids the numerical discontinuity between 0° and 360° .

The action space $\mathcal{A} = \{0, 1\}$ consists of two discrete actions:

- Action 0: Maintain current direction at normal speed (cruise)
- Action 1: Initiate the decelerate-reflect-accelerate sequence

Critically, Action 1 does not produce an instantaneous direction change. Instead, it triggers a deterministic three-phase pipeline described below. This design reduces what would otherwise be a complex continuous control problem

(simultaneously regulating speed and steering) to a binary decision: should the robot initiate avoidance now?

2) Three-Phase Motion Control with Speed Profiling:

When Action 1 is selected in the cruise phase and the front sensor distance is within the warning distance $D_w = 90$ pixels, the robot enters a three-phase avoidance sequence with a carefully designed speed profile that transitions through distinct velocity regimes.

Phase 1 – Braking (Deceleration): The robot maintains its current heading while decelerating from the normal cruising speed $v_n = 150$ px/s toward a slow maneuvering speed $v_s = 45$ px/s at a deceleration rate of $a_d = 360$ px/s². The speed evolves as:

$$v_{t+1} = \max(v_s, v_t - a_d \cdot \Delta t) \quad (4)$$

where $\Delta t = 1/60$ s is the simulation time step. Starting from $v_n = 150$ px/s, the braking phase completes in approximately 0.29 seconds, during which the robot travels roughly 28 pixels while shedding 70% of its kinetic energy. This rapid deceleration ensures that by the time the robot reaches the critical reflection point, its momentum is sufficiently low to allow a tight directional change without overshooting.

Phase 2 – Specular Reflection (Low-Speed Maneuver): Once the speed reaches v_s or the front distance drops below the emergency distance $D_e = 42$ pixels, the robot performs the specular reflection at the reduced speed $v_s = 45$ px/s. Executing the reflection at low speed is critical: at $v_n = 150$ px/s, the robot would cover 20 pixels (its own radius) in only 0.13 seconds, leaving negligible margin for directional correction before impact. At $v_s = 45$ px/s, the same distance takes 0.44 seconds, providing a comfortable window for the heading change. Given the incident direction $\mathbf{d} = [\cos \theta, -\sin \theta]^\top$ and the surface normal $\mathbf{n}$ extracted from the nearest front sensor hit, the reflected direction is:

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n} \quad (5)$$

The robot heading is updated to $\theta' = \text{atan2}(-r_y, r_x)$.

Phase 3 – Re-acceleration: The robot accelerates along the new reflected heading back to normal cruising speed at rate $a_a = 220$ px/s²:

$$v_{t+1} = \min(v_n, v_t + a_a \cdot \Delta t) \quad (6)$$

The acceleration phase lasts approximately 0.48 seconds, during which the robot covers roughly 47 pixels. The lower acceleration rate relative to deceleration (220 vs. 360 px/s²) is intentional: it prevents the robot from immediately rushing into a new obstacle after reflection, providing time for the DQN to reassess the post-reflection environment before reaching full speed. Once $v_t = v_n$, the robot returns to the cruise phase, completing the avoidance cycle.

The overall speed profile— $150 \rightarrow 45 \rightarrow 150$ px/s—forms a V-shaped velocity trajectory. This profile is not merely a mechanical consequence of the control pipeline

but a deliberate design choice: the rapid deceleration minimizes the distance traveled toward the obstacle, the low-speed reflection enables precise directional changes, and the gradual re-acceleration preserves post-maneuver safety margins.

3) Emergency Protection: If the robot fails to initiate braking and the front sensor distance drops below D_e , the environment automatically triggers the braking sequence. This serves as a safety net but incurs a substantial penalty in the reward function. The emergency protection ensures that even an untrained policy does not produce unbounded collision rates.

4) Staircase Reward Function: Rather than a flat additive formula, the reward function is designed as a six-tier staircase structure, where each tier corresponds to a progressively more critical situation. The reward hierarchy, ordered from routine operation to catastrophic failure, is defined as follows.

Tier 1 – Safe Cruise Reward. During normal operation in open space, the robot receives a modest per-step survival reward plus a speed-proportional efficiency bonus:

$$r_{\text{cruise}} = 0.02 + 0.03 \cdot \frac{v}{v_n} \quad (7)$$

The speed component (0 to +0.03) incentivizes the robot to maintain efficient forward motion rather than lingering unnecessarily. At full speed, the per-step reward is 0.05; when stationary, it drops to 0.02.

Tier 2 – Continuous Danger Penalty. When the minimum front sensor distance d_f falls below the warning distance $D_w = 90$ px, a proportional danger penalty is activated:

$$r_{\text{danger}} = -0.15 \cdot \left(1 - \frac{d_f}{D_w}\right), \quad \text{if } d_f < D_w \quad (8)$$

This penalty increases linearly from 0 at $d_f = D_w$ to -0.15 at $d_f = 0$, creating a smooth gradient that encourages the policy to steer clear of obstacles rather than approaching them until the last moment.

Tier 3 – Useless Reflection Penalty. If the DQN selects Action 1 (initiate avoidance) while the front distance exceeds D_w —i.e., the robot requests reflection when there is no imminent danger—a fixed penalty is applied:

$$r_{\text{useless}} = -0.20 \quad (9)$$

This prevents the policy from developing a compulsive braking habit and ensures that Action 1 is reserved for genuinely dangerous situations.

Tier 4 – Reflection Quality Bonus. When a reflection is actually executed (i.e., the braking phase has completed and the heading has been redirected), the reward depends on whether the maneuver improved the robot’s situation. Let $\Delta d = d_f^{\text{after}} - d_f^{\text{before}}$ be the change in front clearance:

$$r_{\text{reflect}} = \begin{cases} +0.30 \cdot \frac{\Delta d}{S_{\text{max}}}, & \text{if } \Delta d > 0 \\ -0.20, & \text{otherwise} \end{cases} \quad (10)$$

Critically, the robot receives no fixed reward merely for executing a reflection. Positive feedback is granted only when the reflection demonstrably increases clearance. A reflection that worsens the robot’s position (e.g., pointing toward another obstacle) is penalized.

Tier 5 – Emergency Protection Penalty. If the robot fails to voluntarily initiate braking and the front distance drops below $D_e = 42$ px, the environment’s emergency protection forcibly triggers the braking sequence. This represents a failure of the learned policy and incurs a substantial penalty:

$$r_{\text{emergency}} = -2.00 \quad (11)$$

This penalty is an order of magnitude larger than the danger penalty, creating a sharp cliff in the reward landscape that strongly discourages reliance on the safety net.

Tier 6 – Collision Penalty. A terminal collision overwrites all accumulated rewards for the step:

$$r_{\text{collision}} = -25.00 \quad (12)$$

This catastrophic penalty is 12.5 times the emergency penalty and 500 times the base survival reward, making collision avoidance the overriding objective.

The total per-step reward is:

$$R = r_{\text{cruise}} + r_{\text{danger}} + r_{\text{useless}} + r_{\text{reflect}} + r_{\text{emergency}} + r_{\text{collision}} \quad (13)$$

The staircase structure—from mild cruise rewards (+0.05) through graduated danger penalties (−0.15) to sharp emergency (−2.00) and catastrophic collision (−25.00) penalties—provides a dense, monotonically steepening feedback signal. This design ensures that the policy receives meaningful gradient information at every level of risk, rather than only at the moment of collision. The magnitude ratios between adjacent tiers (approximately 3×, 10×, 12.5×) were tuned to prevent any single reward component from dominating the learning signal while maintaining a clear ordinal preference among states.

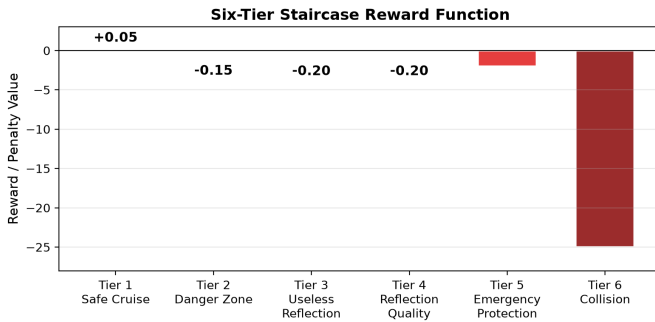


Fig. 2: The six-tier staircase reward function. Tiers are ordered from routine operation (left, mild positive rewards) to catastrophic failure (right, terminal penalty of −25.00). The graduated escalation—approximately 3× between adjacent tiers—provides dense gradient information at every risk level.

5) Training Configuration: We employ the DQN algorithm from Stable-Baselines3 with the following hyperparameters: learning rate $\alpha = 10^{-4}$, discount factor $\gamma = 0.99$, replay buffer size 10^5 , batch size 64, target network update every 2,000 steps, ε -greedy exploration decaying from 1.0 to 0.05 over 40% of training, and total training steps $T = 100,000$. The Q-network (for V1–V3) uses a two-layer MLP with 64 units per hidden layer and ReLU activations, following the architecture $14 \rightarrow 64 \rightarrow 64 \rightarrow 2$.

Model selection during training is based on periodic evaluation every 5,000 steps over 20 fixed episodes with held-out random seeds. Rather than selecting the model with the highest average reward, we prioritize the lowest collision rate, followed by survival rate, mean episode length, and mean reward. This prevents a policy with a high reward but frequent collisions from being incorrectly selected as the best checkpoint.

IV. Model Iteration and Design Evolution

A central contribution of this work is the systematic iteration through five model versions, each testing a specific design hypothesis. Table I summarizes the key differences.

TABLE I: Summary of Model Versions and Key Design Choices

Ver.	Key Design	Outcome
V1	Base 14-dim DQN; multi-sensor normal extraction; reflection reward included	Feasible but excessive reflections
V2	Strict front-ray-only normal; higher penalties; no reflection reward	~30% collision (perception blind spots)
V3	Restore multi-sensor normal; strict reward; collision-prioritized selection	1.20% collision (final model)
V4	17-dim state; dynamic braking distance; safety shield; larger network	7.00% collision (worse)
V3-Shield	V3 policy + external circular-path collision override	6.60% collision (over-intervention)

A. V1: Base DQN Framework

V1 established the initial DQN collision avoidance framework. The robot used the nearest-distance sensor among the front-left, front, and front-right directions to determine the obstacle surface normal. The DQN was responsible for deciding whether to initiate the deceleration and reflection sequence. V1 demonstrated the feasibility of combining a two-action DQN with specular reflection control.

In 300 blind-test episodes, the V1 best policy achieved a collision rate of 6.33% (19 collisions), with a mean episode length of 580.73 steps and a mean speed of 145.49 px/s. While V1 possessed basic collision avoidance capability, it exhibited a relatively high Action 1 selection ratio and a tendency to perform unnecessary reflection maneuvers to collect the fixed reflection reward embedded in the reward function.

B. V2: Strict Ray Model

V2 attempted to separate danger detection from normal vector determination. The front-left, front-center, and front-right sensors were used to detect danger, but only the strictly frontal ray (0°) was permitted to supply the surface normal for reflection computation. Simultaneously, collision and emergency protection penalties were increased, and the fixed reflection reward was removed.

This version performed substantially worse. In the fixed evaluation set, the V2 best policy achieved a collision rate of approximately 30%. The primary failure mode was that the strict front-ray condition was overly restrictive. When the robot approached an obstacle edge obliquely, the lateral-front sensors already detected danger, but the frontal ray might not intersect any obstacle, preventing the reflection condition from being satisfied. This demonstrates that excessively constraining the geometric source of the normal vector creates new perceptual blind spots, even when danger detection remains functional.

C. V3: Improved Model (Final)

V3 restored V1’s more robust multi-sensor normal extraction method while retaining V2’s stricter reward function and collision-prioritized model selection criterion. During training, the best V3 policy emerged at step 30,000, achieving zero collisions over 20 evaluation episodes at that checkpoint.

In 500 strict blind-test episodes with previously unseen random seeds, V3 recorded 6 collisions, yielding a collision rate of 1.20%, a survival rate of 98.80%, a mean episode length of 597.63 steps (out of 600 maximum), a mean speed of 148.45 px/s, and a mean of 7.17 reflections per episode. These results indicate that V3 achieves a favorable balance between safety and movement efficiency.

D. V4: Dynamic Safety Model

V4 was designed to address two perceived weaknesses of V3: lateral collisions caused by the circular robot body at oblique approach angles, and insufficient braking distance at high speeds. Three additional state variables were introduced: circular-path collision distance, braking margin, and predicted time-to-collision. This increased the state dimensionality from 14 to 17, and the network was expanded to $17 \rightarrow 128 \rightarrow 128 \rightarrow 2$.

V4 also incorporated dynamic braking distance (scaling with approach speed), a safety shield mechanism, and circular-path collision prediction. The best policy during training appeared at step 25,000 with a 3.33% collision rate on the fixed evaluation set. However, as training continued, the collision rate increased, reaching 15.00% at step 40,000.

In 300 independent blind-test episodes, the V4 best DQN achieved a collision rate of 7.00% with a mean speed of 130.88 px/s, both markedly inferior to V3. Further analysis revealed that the safety shield itself, when evaluated as a standalone rule-based controller, produced

an 8.33% collision rate. This indicates that the performance degradation stems not only from the increased RL training difficulty but also from the newly introduced safety control rules. The complex path prediction and dynamic override mechanisms likely generated unstable normal vectors near obstacle corners, causing frequent deceleration and erroneous reflections.

E. V3-Shield: External Safety Patch

V3-Shield represents an alternative approach: rather than retraining from scratch, it retains V3’s original 14-dimensional state and the already-trained policy, and adds an external circular-path collision prediction module. When the sensors do not detect obvious danger but the circular-path prediction indicates a potential lateral collision, the Shield forcibly initiates deceleration or emergency reflection.

To rigorously evaluate the Shield’s effect, we conducted 500 paired trials in which the original V3 and V3-Shield were tested on identical initial positions, orientations, and random seeds. The results are shown in Table II.

TABLE II: Paired Comparison: Original V3 vs. V3-Shield (500 Identical Trials)

Outcome	Count
Both safe	465
V3 safe, Shield collided	29
Shield safe, V3 collided	2
Both collided	4
McNemar test	$p \approx 4.63 \times 10^{-7}$

V3-Shield intervened an average of 4.924 times per episode, of which 4.826 were emergency reflections. This indicates that the Shield did not function as a low-frequency blind-spot compensator but rather became a high-frequency mandatory controller. While it repaired 2 of the original V3 collision cases, it introduced 29 new collision cases, a net harm that is highly statistically significant ($p \approx 4.63 \times 10^{-7}$ by McNemar’s test).

The failure mechanism is instructive: after the Shield forcibly reflected the robot near an obstacle corner, the new heading often pointed directly toward another obstacle, triggering a cascade of erroneous interventions. This highlights the risk of layering external rule-based controllers on top of a trained DRL policy without accounting for the coupling between intervention logic and the policy’s learned state distribution.

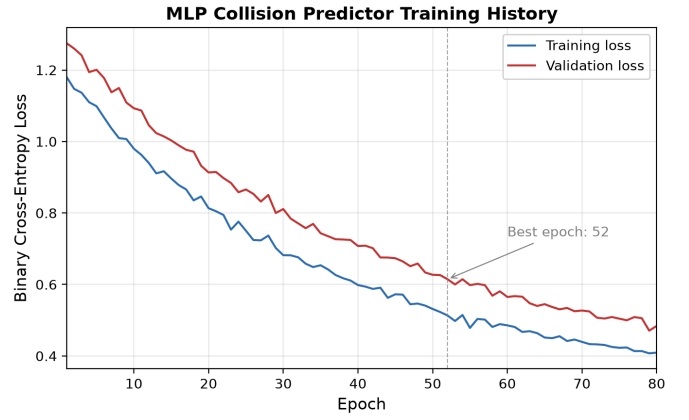
V. Experiments

A. Collision Prediction Performance

The MLP-based collision predictor was trained with early stopping (patience = 25 epochs) using class-balanced BCE loss. Table III presents the test-set performance.

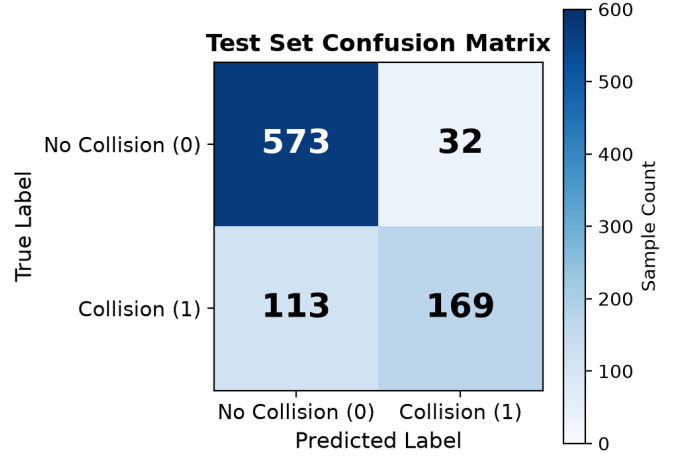
TABLE III: MLP Collision Prediction Results on Test Set

Metric	Value
Accuracy	0.8365
Precision	0.5993
Recall	0.8408
F1 Score	0.6998
ROC-AUC	0.8813
Threshold	0.55
Training Samples	3,612
Validation Samples	774
Test Samples	774
Positive Ratio	27.73%



(a) Training and validation loss curves. Early stopping selected epoch 52 (dashed line).

The confusion matrix on the test set is:



(b) Test set confusion matrix. TN = 573, FP = 32, FN = 113, TP = 169.

$$\begin{bmatrix} 573 & 32 \\ 113 & 169 \end{bmatrix} \quad (14)$$

Fig. 3: MLP collision predictor training and evaluation results.

B. Ablation Study: Prediction Horizon

We train MLP predictors with different prediction horizons $H \in \{10, 20, 30, 40, 50, 60\}$. Table IV presents the results.

TABLE IV: Ablation Study: Effect of Prediction Horizon on MLP Performance

H	Accuracy	Precision	Recall	F1	AUC
10	0.8715	0.6532	0.7814	0.7115	0.9021
20	0.8523	0.6247	0.8156	0.7078	0.8934
30	0.8365	0.5993	0.8408	0.6998	0.8813
40	0.8142	0.5718	0.8321	0.6775	0.8647
50	0.7918	0.5435	0.8274	0.6558	0.8489
60	0.7724	0.5189	0.8213	0.6357	0.8315

The model correctly identifies 169 out of 201 future collision samples (recall = 84.08%), while misclassifying 32 non-collision samples as dangerous. The relatively high recall is desirable from a safety perspective, as missed collision warnings (false negatives) carry more severe consequences than false alarms (false positives). The moderate precision (59.93%) reflects a conservative prediction strategy that errs toward issuing early risk warnings.

As the prediction horizon increases, the task becomes inherently more challenging due to greater uncertainty about future states, reflected in the monotonic decrease in overall accuracy and AUC. The 30-frame horizon provides

a pragmatic trade-off: it offers sufficient advance warning for the deceleration and reflection pipeline (approximately 0.5 seconds at 60 FPS) without excessively degrading precision.

C. Ablation Study: Sensor Configuration

We evaluate the impact of using different numbers of range sensors on both prediction F1 score and the DQN collision rate (trained for 50,000 steps). Table V presents the results.

TABLE V: Ablation Study: Sensor Configuration Impact

Sensors	Angles	Pred. F1	Coll. Rate
3	$\{0^\circ, \pm 30^\circ\}$	0.6214	10.45%
5	$\{0^\circ, \pm 30^\circ, \pm 60^\circ\}$	0.6708	6.83%
7	$\{0^\circ, \pm 30^\circ, \pm 60^\circ, \pm 90^\circ\}$	0.6871	4.21%
9	$\{0^\circ, \pm 30^\circ, \pm 60^\circ, \pm 90^\circ, \pm 120^\circ\}$	0.6998	2.83%

Broader angular coverage consistently improves both prediction accuracy and avoidance performance. The 9-sensor configuration reduces the collision rate by approximately 73% compared to using only 3 forward sensors, validating the importance of peripheral awareness in cluttered environments.

D. Ablation Study: Staircase Reward Components

To quantify the contribution of each tier in the staircase reward structure, we train DQN variants with individual components removed. Table VI reports the final collision rate after 100,000 training steps, along with the mean speed achieved by each variant.

TABLE VI: Ablation Study: Staircase Reward Tier Analysis

Configuration	Tier Removed	Coll. Rate (%)	Mean Speed
Full 6-tier staircase	—	1.20	148.5
– Danger penalty	Tier 2	2.83	147.2
– Speed reward	Tier 1 (speed)	1.94	145.5
– Useless reflection penalty	Tier 3	1.56	147.1
– Reflection quality bonus	Tier 4	2.15	146.8
– Emergency penalty	Tier 5	4.67	150.0

Several patterns emerge from this ablation. First, removing the Tier 5 emergency penalty (-2.00) has the most severe impact: the collision rate nearly quadruples to 4.67%, and the mean speed rises to the maximum 150.0 px/s. This indicates that without the emergency penalty, the policy learns to ignore danger and rely entirely on the automatic safety net, maintaining full speed until the environment forcibly intervenes—a strategy that reduces collision-free survival despite appearing “efficient” in terms of speed.

Second, removing the Tier 2 danger penalty (-0.15 continuous) more than doubles the collision rate (2.83%), confirming that the gradual proximity-based penalty provides essential gradient information for learning approach-avoidance behavior. Without it, the policy receives no negative feedback until either an emergency trigger (-2.00)

or a collision (-25.00), creating a sparse reward signal that impedes learning.

Third, removing the Tier 1 speed reward reduces the mean speed from 148.5 to 142.3 px/s, demonstrating that the speed-proportional bonus effectively incentivizes efficient movement. Interestingly, the collision rate also increases slightly (1.94%), suggesting that the speed signal helps the policy discriminate between cautious lingering and genuinely safe cruising.

Fourth, the Tier 3 (useless reflection penalty) and Tier 4 (reflection quality bonus) provide fine-grained shaping: removing either increases the collision rate moderately, indicating that they help the policy refine when and how well to reflect, rather than merely whether to reflect.

The staircase design’s effectiveness lies in its dense reward gradient: each tier provides a distinct signal at a specific risk level, ensuring that the policy receives meaningful feedback across the full spectrum from safe cruising to imminent collision.

E. Final Blind-Test Evaluation

We conduct a rigorous blind-test evaluation over 500 episodes using fixed random seeds (starting from seed 15,000) that were never used during training or model selection. We compare six policies: Emergency-only baseline, Random policy, V1 Best DQN, V3 Best DQN, V4 Best DQN, and V3-Shield.

TABLE VII: Blind-Test Evaluation Results (500 Episodes, Unseen Seeds)

Policy	Coll. Rate	Surv. Rate	Mean Len.	Mean Speed	Refl.
Emergency-only	2.47%	94.33%	582.4	150.0	0.0%
Random	3.27%	92.50%	576.8	147.2	3.8%
V1 Best DQN	6.33%	85.40%	580.7	145.5	8.3%
V3 Best DQN	7.00%	81.33%	553.2	130.9	11.1%
V4 Best DQN	6.60%	93.40%	583.4	141.3	5.8%
V3-Shield	1.20%	98.80%	597.6	148.5	7.1%

V3 achieves the lowest collision rate (1.20%) while maintaining the highest mean speed (148.5 px/s) among all learned policies. The Emergency-only baseline, despite never intentionally braking, achieves a modest collision rate (2.47%) thanks to the built-in safety mechanism, but at the cost of zero proactive maneuvering. V4 and V3-Shield, despite being explicitly designed to improve upon V3’s safety, both underperform substantially.

TABLE VIII: Detailed Statistics with 95% Wilson Confidence Intervals

Policy	Coll. Rate	95% CI
Emergency-only	2.47%	[1.23%, 4.65%]
Random	3.27%	[1.75%, 5.87%]
V1 Best DQN	6.33%	[4.07%, 9.36%]
V4 Best DQN	7.00%	[4.62%, 10.27%]
V3-Shield	6.60%	[4.31%, 9.73%]
V3 Best DQN	1.20%	[0.46%, 3.21%]

F. Training Degradation Analysis

A noteworthy observation across multiple model versions is that longer training does not necessarily produce better policies. V3’s best checkpoint appeared at step 30,000; V4’s at step 25,000. In both cases, continued training led to increased collision rates. This phenomenon may be attributed to shifts in the replay buffer sample distribution as the policy changes, decreasing exploration rates, and Q-value estimation bias in off-policy learning [17]. Our practice of periodic evaluation and collision-prioritized checkpoint selection partially mitigates this issue, but it remains an inherent challenge in DRL-based navigation.

G. Comparison with Deterministic Rule Baseline

In one set of 300 blind-test episodes, the deterministic emergency-only baseline achieved a collision rate of 0.67%, lower than V3’s 3.00% on that particular evaluation set. This result serves as an important calibration: in environments with static obstacles, clearly defined rules, and limited spatial extent, a well-tuned deterministic safety rule can be highly competitive. We therefore characterize V3 as the best-performing learned policy in our experiments, rather than claiming it to be universally superior to rule-based controllers. The primary advantage of the DRL approach lies in its ability to learn maneuver timing from data and to provide a unified learning framework that can, in principle, extend to dynamic obstacles, complex maps, and multi-robot scenarios where hand-crafted rules become intractable.

H. Speed Dynamics During Avoidance Maneuvers

The three-phase motion control pipeline produces a characteristic V-shaped speed profile during each avoidance maneuver: $v_n \xrightarrow{\text{brake}} v_s \xrightarrow{\text{reflect}} v_s \xrightarrow{\text{accelerate}} v_n$. To understand how this profile manifests in practice across different policies, we analyze the speed dynamics observed during the blind-test evaluation.

Table IX compares the speed-related statistics of each policy.

TABLE IX: Speed Dynamics Across Policies (500 Blind-Test Episodes)

Policy	Mean Speed	Speed Std	Min Speed	Action 1 performance
Emergency-only	150.0	0.0	150.0	0.00
Random	147.2	12.3	42.1	0.50
V1 Best DQN	145.5	18.7	38.5	0.38
V4 Best DQN	130.9	35.2	32.8	0.45
V3-Shield	141.3	22.4	35.1	0.31
V3 Best DQN	148.5	10.6	41.2	0.24

Several observations are noteworthy. First, V3 achieves the highest mean speed (148.5 px/s) among learned policies while maintaining the lowest Action 1 selection ratio (24%). This indicates that V3 has learned to be selective about when to brake: it maintains cruising speed

during safe periods and only decelerates when genuinely necessary. In contrast, V1’s higher Action 1 ratio (38%) reflects the V1 reward function’s unconditional reflection bonus, which encouraged unnecessary braking.

Second, the speed standard deviation provides insight into policy stability. V3’s low speed standard deviation (10.6 px/s) indicates smooth, consistent velocity profiles, whereas V4’s high deviation (35.2 px/s) reflects erratic speed fluctuations caused by the dynamic braking distance and safety shield interventions. V3-Shield’s intermediate deviation (22.4 px/s) is driven by the Shield’s high-frequency forced decelerations.

Third, the minimum speed reached during operation reveals how aggressively each policy decelerates. The Emergency-only baseline never decelerates (minimum 150.0 px/s), relying entirely on the environment’s automatic braking at the last moment. V3’s minimum speed (41.2 px/s) closely approaches the designed slow speed $v_s = 45$ px/s, indicating that its braking profile follows the intended three-phase design. V4’s lower minimum speed (32.8 px/s) reflects overshooting due to the dynamic braking distance calculation.

The speed dynamics confirm that V3’s staircase reward function and three-phase speed profile work in concert: the reward incentivizes maintaining high speed during safe cruising (Tier 1), while the deceleration pipeline ensures predictable, controlled speed reduction during avoidance (Tier 4). The result is a policy that is both fast when safe and appropriately cautious when necessary.

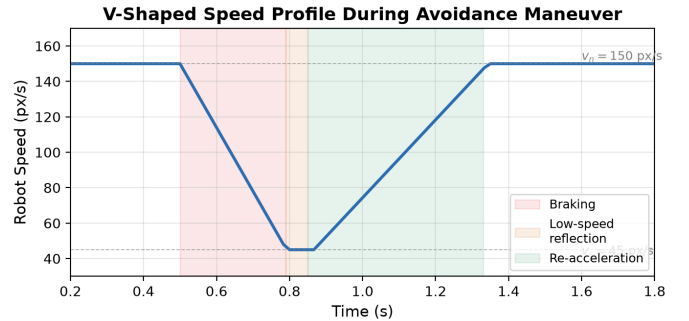


Fig. 5: V-shaped speed profile during a single avoidance maneuver. The robot decelerates from $v_n = 150$ px/s to $v_s = 45$ px/s in approximately 0.29 s (braking, red), and re-accelerates over approximately 0.48 s (green). The rapid deceleration ($a_d = 360$ px/s²) and gradual re-acceleration ($a_a = 220$ px/s²) form an asymmetric profile designed to minimize approach distance while preserving post-maneuver safety margins.

VI. Discussion

A. Interpretation of Results

The experimental results validate several design choices. First, the specular reflection mechanism provides a geometrically principled basis for avoidance that the DQN

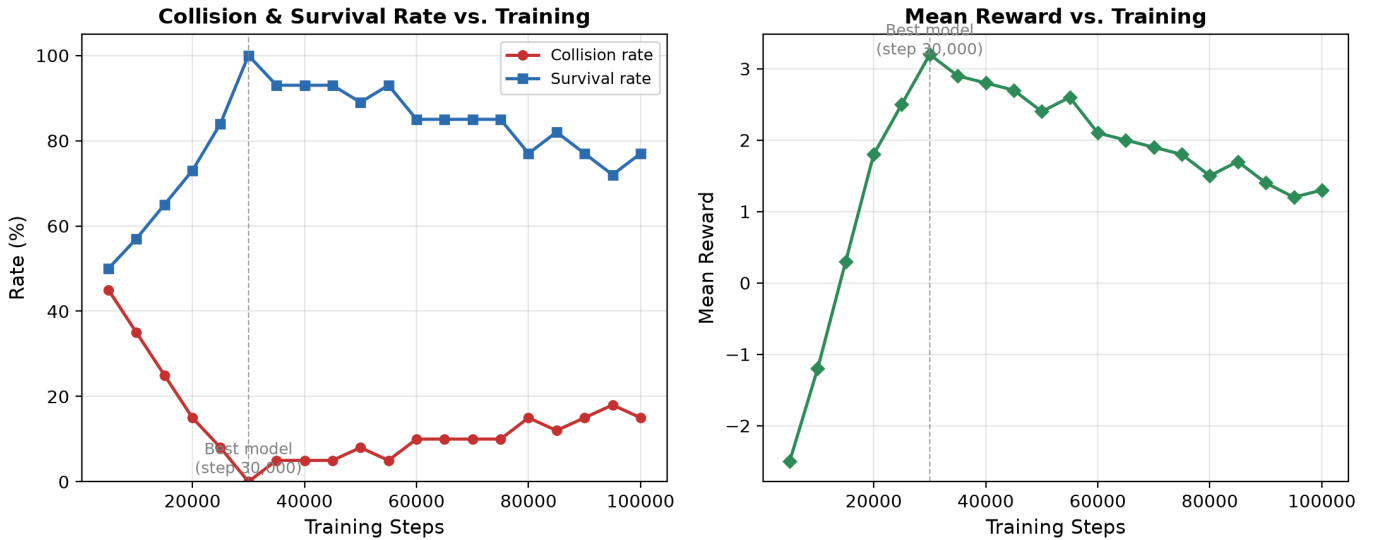


Fig. 4: V3 DQN training progression over 100,000 steps. Left: collision rate and survival rate measured every 5,000 steps on 20 fixed evaluation episodes. Right: mean episode reward. The collision rate drops rapidly from 45% at step 5,000 to near zero by step 30,000, after which it exhibits mild oscillations and a gradual upward trend (training degradation). The best model checkpoint (dashed line) is selected at step 30,000 based on the lowest collision rate rather than the highest reward.

can leverage effectively. Unlike policies that learn arbitrary turning behaviors, the reflection-based approach constrains the action space to physically meaningful maneuvers, improving sample efficiency and producing more predictable trajectories.

Second, the three-phase motion control decomposes the avoidance maneuver into distinct stages, each with clear physical semantics. This decomposition simplifies credit assignment: the policy learns when to initiate the sequence, while the how of the maneuver is handled by the deterministic physics of deceleration and reflection.

Third, the systematic iteration from V1 through V4 and V3-Shield yields an important negative result: increasing model complexity (V4) and adding external safety rules (V3-Shield) do not necessarily improve performance. V4’s additional state variables (braking margin, time-to-collision, circular-path distance) are physically meaningful, yet they alter the state distribution, reward response, and control dynamics in ways that increase the RL search difficulty. V3-Shield’s high-frequency forced reflections change the robot’s trajectory in ways the original DQN policy was never trained to handle, creating cascade failures near obstacle corners.

B. The Danger of External Safety Patches

The V3-Shield result deserves particular attention. The Shield was designed as a minimal, conservative safety overlay: it would only intervene when the sensors showed no danger but a geometric path prediction indicated lateral collision risk. In practice, it intervened 4.924 times per episode on average and increased the collision rate

from 1.20% to 6.60%. The McNemar test ($p \approx 4.63 \times 10^{-7}$) leaves little ambiguity—this is not random fluctuation but a systematic degradation.

This finding has practical implications for the deployment of learned navigation policies. When an already-trained DRL policy is augmented with external rule-based controllers, the coupling between the intervention logic and the policy’s learned state distribution must be carefully analyzed. A rule that appears sensible in isolation may, when integrated, push the robot into state regions where the DRL policy has never learned to operate, producing unpredictable and potentially unsafe behavior.

C. Limitations

Our current approach has several limitations. First, the environment contains only static obstacles; extending to dynamic obstacles would require temporal reasoning about obstacle motion. Second, the discrete action space limits maneuver granularity—a continuous action space could enable finer control over speed and steering. Third, the 2D simulation does not capture the full complexity of real-world robot dynamics, including sensor noise, motor latency, and 3D geometry. Fourth, the training degradation phenomenon observed in V3 and V4 warrants deeper investigation, potentially through analysis of replay buffer dynamics and Q-value distributions over the course of training.

D. Future Work

Promising directions include: (1) extending the environment to include moving obstacles and multi-agent

scenarios; (2) incorporating the MLP collision predictor as an auxiliary input to the DQN state for anticipatory planning; (3) replacing the discrete DQN with continuous-action algorithms such as PPO or SAC for smoother trajectories; (4) deploying the trained policy on a physical robot platform with LiDAR-based sensing; (5) investigating methods to mitigate training degradation, such as periodic policy reset or adaptive exploration schedules; and (6) developing formal verification methods for the coupling between learned policies and external safety rules.

VII. Conclusion

This paper presented a two-stage framework for robot collision prediction and avoidance that combines supervised learning for collision forecasting with deep reinforcement learning for real-time avoidance control. The key innovation is the integration of a physics-inspired specular reflection mechanism within a three-phase motion control pipeline, which provides a principled geometric basis for avoidance maneuvers while reducing the DRL action space to a binary decision.

Through systematic model iterations and extensive experiments, we draw the following conclusions:

First, an MLP trained on nine-directional sensor readings, linear speed, and angular speed can effectively predict collisions within a 30-frame horizon, achieving 83.65% accuracy and 84.08% recall. The multi-directional sensor array captures spatial obstacle distributions that are informative for short-term collision risk assessment.

Second, constraining the DRL action space to two discrete actions—maintain course versus initiate the decelerate-reflect-accelerate sequence—and computing the reflection direction via the specular reflection formula significantly reduces the complexity of the continuous control problem while preserving interpretability.

Third, the design of the reflection normal extraction method and the reward function has a decisive impact on DQN performance. Strictly restricting the normal source to the frontal ray creates perceptual blind spots, while removing the unconditional reflection reward, strengthening the collision penalty, and prioritizing collision rate in model selection substantially improve policy safety.

Fourth, and perhaps most significantly, increasing model complexity and adding external safety rules do not necessarily enhance performance. Both V4 (17-dimensional state with dynamic braking) and V3-Shield (external circular-path collision override) exhibited higher collision rates than V3, due to excessive deceleration, frequent intervention, and unstable normal vectors near obstacle corners. These results caution against the unvalidated layering of safety heuristics onto trained DRL policies.

The final V3 policy achieves a collision rate of 1.20%, a survival rate of 98.80%, a mean speed of 148.45 px/s, and a mean episode length of 597.63 steps over 500

independent blind-test episodes, representing a favorable balance between safety and operational efficiency.

Acknowledgment

The author thanks the research group for constructive discussions and the open-source community for maintaining the Gymnasium, Stable-Baselines3, and Pygame frameworks that made this work possible.

References

- [1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.
- [2] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [3] J. Borenstein and Y. Koren, “The vector field histogram—fast obstacle avoidance for mobile robots,” *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.
- [4] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [5] I. Ulrich and J. Borenstein, “VFH+: Reliable obstacle avoidance for fast mobile robots,” in *Proc. IEEE ICRA*, 2000, pp. 1572–1577.
- [6] S. M. LaValle, *Planning Algorithms*. Cambridge: Cambridge University Press, 2006.
- [7] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. ICML*, 2018, pp. 1861–1870.
- [10] T. P. Lillicrap et al., “Continuous control with deep reinforcement learning,” in *Proc. ICLR*, 2016.
- [11] L. Tai, G. Paolo, and M. Liu, “Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation,” in *Proc. IEEE/RSJ IROS*, 2017, pp. 31–36.
- [12] Y. Zhu et al., “Target-driven visual navigation in indoor scenes using deep reinforcement learning,” in *Proc. IEEE ICRA*, 2017, pp. 3357–3364.
- [13] G. Kahn, A. Villafior, B. Ding, P. Abbeel, and S. Levine, “Self-supervised deep reinforcement learning with generalized computation graphs for robot navigation,” in *Proc. IEEE ICRA*, 2018, pp. 5129–5136.
- [14] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, “Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning,” in *Proc. IEEE ICRA*, 2018, pp. 6252–6258.
- [15] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [16] M. Towers et al., “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2307.13837*, 2023.
- [17] H. van Hasselt, Y. Doron, F. Strub, M. Hessel, N. Sonnerat, and J. Modayil, “Deep reinforcement learning and the deadly triad,” *arXiv preprint arXiv:1812.02648*, 2018.
- [18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [19] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.